Do Prompt-Lint Warnings Predict Real Failures?

Empirical validation and prompt-optimization-based repair of conflicts in LLM system prompts

Why we are doing it

AI apps are controlled by a system prompt, a list of rules, and rules can clash. For example, "Always answer every question" and "Never reveal discount codes" disagree when a user asks for a discount code. Prompt linters flag clashes like this, but their warnings are heuristics, and nobody has checked how often a warning means a real failure. If warnings are mostly false alarms, developers waste time on them. If clean prompts still fail, linters give false confidence. Our project measures this, so developers know how far to trust prompt linters.

Problem statement

Design a framework that measures whether the warnings of prompt linters predict real LLM failures, and repairs flagged system prompts using prompt-engineering and programmatic prompt-optimization methods. The framework compares a rule-based linter and an LLM-based linter on 40 system prompts, tests them on at least two LLMs, and quantifies how repair changes the failure rate and the accuracy on normal requests.

What we are doing

We run two linters (one rule-based, one LLM-based) on 40 system prompts: 16 real, 16 with planted conflicts and 8 clean controls. Every prompt gets 4 test scenarios, run on 2 LLMs and scored by code checks and an LLM-as-Judge. We then repair flagged prompts with a lint-guided rewrite, prompt-pattern variants and one optimizer (DSPy or TextGrad), and re-test to see whether failures drop without breaking normal answers.

Approach

1. **Collect prompts:** 40 system prompts across e-commerce, education, finance, healthcare administration and customer support: real (public GitHub repositories, source and licence logged), planted (direct contradiction, scope clash, conditional with no else, priority ambiguity) and clean controls.
2. **Lint:** run the rule-based linter and the LLM-based linter (schema-validated JSON report) on every prompt and record each warning with its type.
3. **Test:** 4 zero-shot scenarios per prompt (a normal request, two rule probes, a pressure request) on 2 LLMs at two temperatures; for conflicts, the rule that should win is written down.
4. **Label:** mark pass or fail with code checks (secret leaked, JSON valid) where possible and an LLM-as-Judge otherwise; log BPE token counts and latency.
5. **Repair:** lint-guided rewrite, priority-order, delimiter and CoT self-check variants, and one optimizer; re-run the same tests plus normal requests to check nothing broke.
6. **Analyse:** warning precision P(fail | flagged), miss rate P(fail | not flagged), linter comparison (precision, recall, F1), failure drop after repair, and repair cost in LLM calls and tokens.

Tech stack

Purpose	Tool
Language and environment	Python 3, Google Colab
Version control and data	GitHub (prompts, tests, results, licence log)
Rule-based linter	rule-audit
LLM-based linter, judge, repair and test models	Gemini API (Google AI Studio, google-genai) plus one model from a different family (to be finalised)
Structured output and chaining	Pydantic, LangChain output parsers
Prompt optimization	DSPy or TextGrad (to be finalised)
Analysis and dashboard (planned)	pandas, matplotlib, Streamlit
Report	IEEE format

Limitations: with 40 prompts, results are indicative, not conclusive, and findings may not generalise to all prompt linters.